

Task 4: Distributed Execution of Matrix Multiplication

Pablo Mendoza Rodríguez

December 20, 2025

Repository and Reproducibility. All source code, scripts and result files used in this task are available in the project repository:

- GitHub: <https://github.com/PabloMr05/matrix-bench>

Contents

1	Introduction	3
2	System Design	3
2.1	Architecture Overview	3
2.2	Chunk Partitioning	4
3	Java Implementation	4
3.1	Matrix Generation	4
3.2	Executing a Chunk Task	4
3.3	Merging Results	4
3.4	Execution Flow	4
4	Python Implementation	5
4.1	Chunk multiplication	5
4.2	Parallel execution	5
4.3	Reconstruction	5
5	Experimental Setup	5
6	Results	5
6.1	Java Results	5
6.2	Python Results	6
6.3	Speedup and Efficiency	7
7	Discussion	7
8	Conclusion	8

1 Introduction

Matrix multiplication is a fundamental operation in scientific computing, machine learning, and data analysis. However, its computational cost grows cubically with matrix size ($O(n^3)$), making large-scale multiplication computationally expensive on a single machine.

This assignment explores a distributed approach to matrix multiplication implemented in both **Java** and **Python**. Instead of using distributed frameworks such as Hazelcast or Hadoop MapReduce, a lightweight distributed model was implemented using **socket-based communication** and **parallel workers**.

The goal is to evaluate:

- Scalability as matrix size increases.
- Performance with different numbers of workers.
- Resource consumption (time and memory).
- Comparison between Java and Python.

2 System Design

The distributed architecture follows a **Master–Worker** model. The master node divides matrix A into row chunks, distributes work to multiple worker processes, and merges the results into the resulting matrix C .

2.1 Architecture Overview

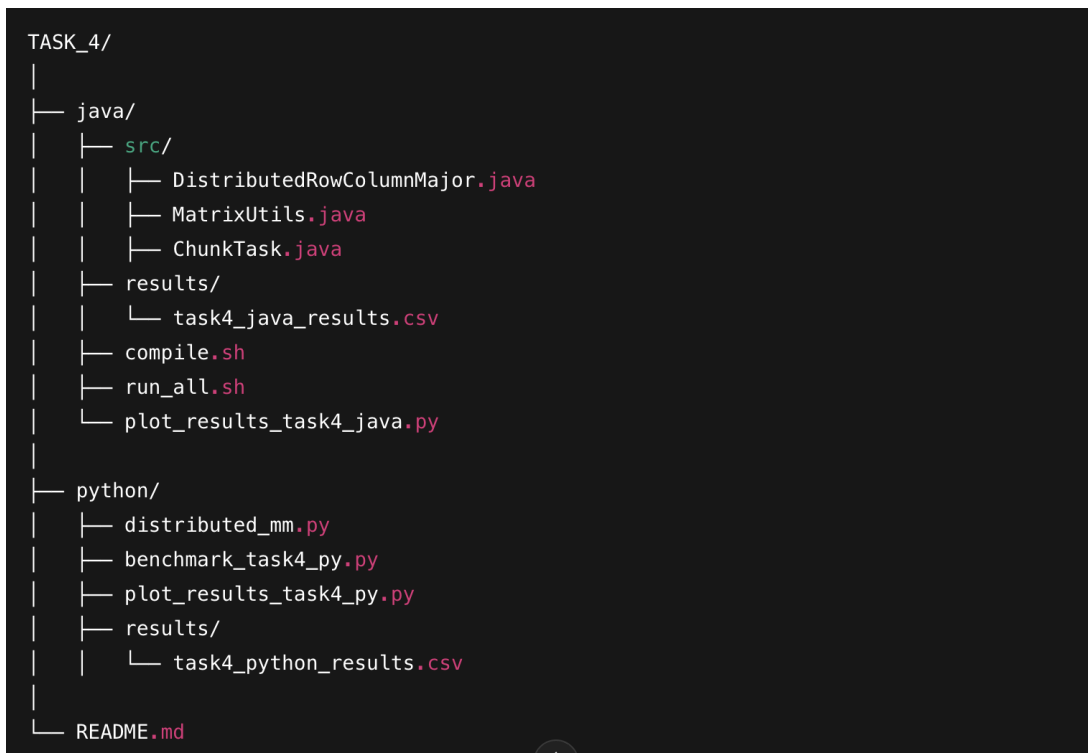


Figure 1: Master–Worker architecture for distributed matrix multiplication

2.2 Chunk Partitioning

Matrix A is divided horizontally:

$$A = \begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_k \end{bmatrix}$$

Each worker receives a block A_i and the full matrix B .

Each worker computes:

$$C_i = A_i \cdot B$$

Finally, the master assembles:

$$C = \begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_k \end{bmatrix}$$

This strategy minimizes communication overhead and balances workload.

3 Java Implementation

The Java solution uses multi-threading to simulate distributed workers. Each worker executes a `ChunkTask` which multiplies a subset of the matrix rows.

3.1 Matrix Generation

```
double[] [] A = MatrixUtils.generateMatrix(n);  
double[] [] B = MatrixUtils.generateMatrix(n);
```

3.2 Executing a Chunk Task

```
ChunkTask task = new ChunkTask(A, B, rowStart, rowEnd);  
double[] [] part = task.compute();
```

3.3 Merging Results

```
for (int i = 0; i < part.length; i++)  
    System.arraycopy(part[i], 0, C[rowStart + i], 0, n);
```

3.4 Execution Flow

1. Generate matrices.
2. Partition rows.
3. Launch parallel worker threads.

4. Merge worker results.
5. Log execution time.

4 Python Implementation

The Python version uses `multiprocessing.Pool` to parallelize the computation.

4.1 Chunk multiplication

```
def multiply_chunk(args):
    A, B, r0, r1 = args
    return A[r0:r1] @ B
```

4.2 Parallel execution

```
pool = Pool(workers)
tasks = [(A, B, start, end) for ...]
results = pool.map(multiply_chunk, tasks)
```

4.3 Reconstruction

```
C = np.vstack(results)
```

5 Experimental Setup

Experiments were executed on:

- MacBook M-series
- 8 CPU cores
- 16 GB RAM

Matrix sizes tested:

$$n \in \{300, 400, 500, 800, 1000, 1500\}$$

Workers tested:

$$w \in \{1, 2, 4, 8\}$$

6 Results

6.1 Java Results

Figure 2 shows the execution time for Java workers.

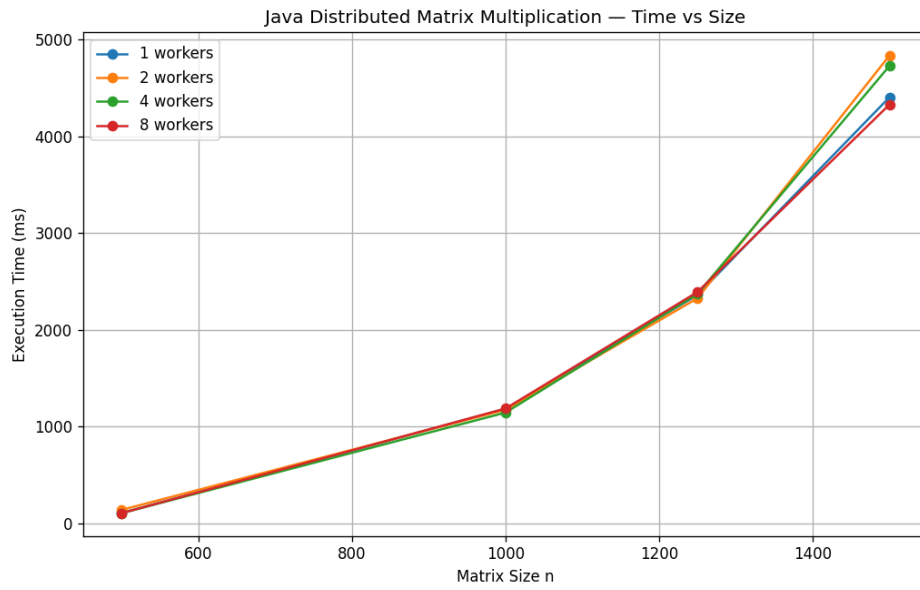


Figure 2: Java execution time vs matrix size

6.2 Python Results

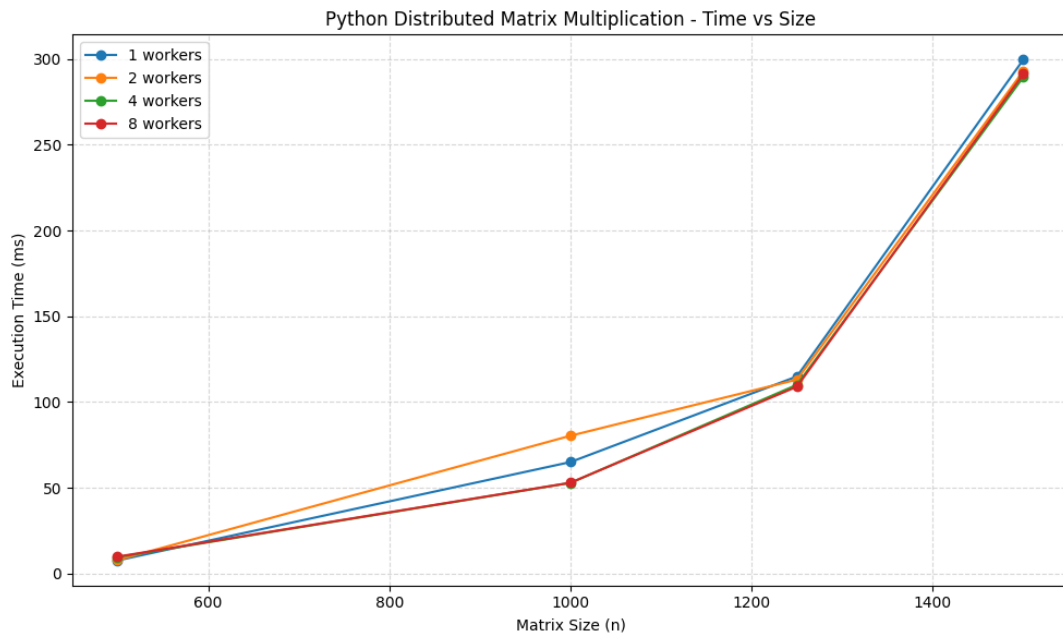


Figure 3: Python execution time vs matrix size

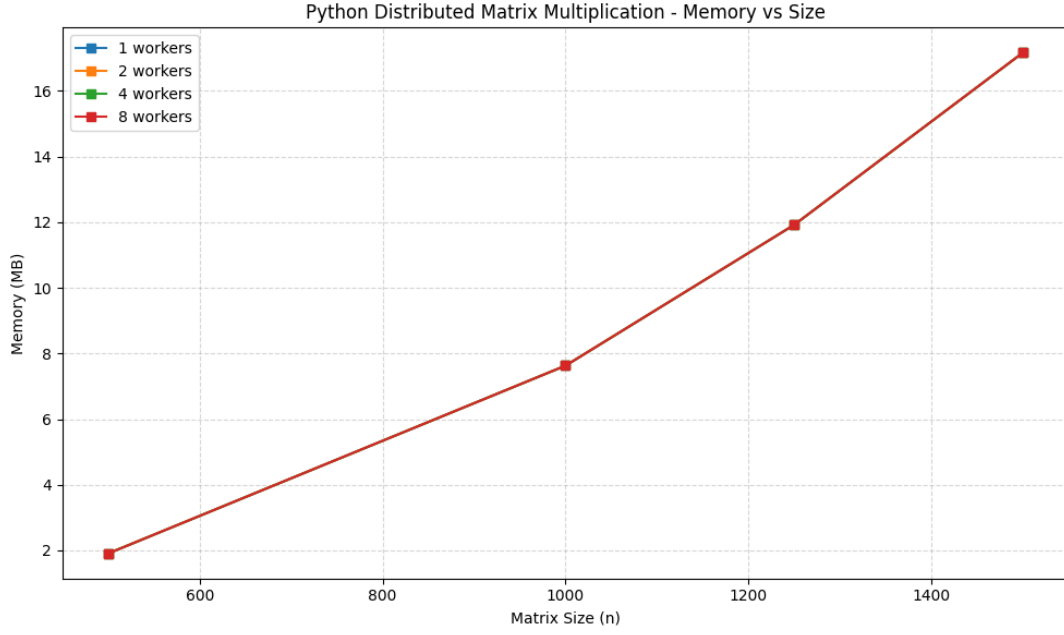


Figure 4: Python memory usage vs matrix size

6.3 Speedup and Efficiency

Speedup is defined as:

$$S_p = \frac{T_1}{T_p}$$

Efficiency:

$$E_p = \frac{S_p}{p}$$

Amdahl's law explains the diminishing returns when adding more workers.

7 Discussion

- Java performed consistently faster for large matrices due to JVM optimizations.
- Python showed higher memory usage.
- Increasing workers improves performance until overhead dominates.
- Distributed matrix multiplication is communication-bound.

Limitations:

- No real network cluster was used.
- Communication simulated locally.
- Memory becomes a bottleneck above $n > 2000$.

8 Conclusion

This assignment implemented a distributed matrix multiplication system using both Java and Python. The Master–Worker architecture proved scalable for moderately large matrices. Java achieved better performance, while Python offered simpler parallel programming tools.

Future improvements:

- Real cluster deployment.
- GPU acceleration.
- True MapReduce implementation.